Logan Sutton
ID: 11798384
Cpts 360
PA 3

I started my implementation by first setting up a basic kernel module project. The goal of this was just to establish a basic foundation to build upon. The module would just log that it was created and log when destroyed. Next, I started developing the procfile directory and status file. I set both up upon initialization of the module, and both are destroyed when the module is removed. Then, I set up the write functionality for the module.

The write functionality works as follows: when the proc/kmlab/status file is written to, the procfile_write() function first clears the internal buffer, checks that the length is acceptable, and then copies the input from the user (using copy_from_user()), which is the process ID. Then, I convert the string process ID into an integer and handle errors accordingly. If no errors occur, I then allocate memory for the proc_list struct, which will be inserted into the kernel list. I initialze the struct and then insert it at the tail of the kernel list using list_add_tail().

Then, I implemented the userapp.c file. This was relatively straightforward. I utilized sprintf() to format the echo command and then used a system call to execute it.

Next, I implemented the kernel timer and work function. The timer continuously calls back to itself every 5 seconds, and within each callback, if the list isn't empty, I schedule any work to be done and then print the contents of the list (if not empty). The work is handled by the work_function(). This function safely iterates through the list using list_for_each_entry_safe() and updates the CPU time for each process in the list (using the given get_cpu_time() function). If the process is done, it is safely removed from the list, and the memory is freed. The work function also makes use of spinlocks to prevent interruption.

Finally, I implemented the read functionality. When a procfile_read() is called, I first check the offset for infinite loops and handle it accordingly. Then, I allocate enough memory for a custom buffer to store the formatted string output. This is done by counting how many items are in the list (utilizing list_for_each_entry() and spinlocks) and then using the scnprintf() function. I used to function to keep track of how many bytes were used in the buffer, as to properly update the offset at the end of the function. Then, I copy the buffer to the user. All allocated memory is freed, and any errors are handled.

When the kernel module is removed, all created assets (proc/kmlab, proc/kmlab/status, timer, workqueue, entries in list) are all properly destroyed/freed from memory.

Screenshots from my implementation are shown below:

From kmlab_test3.sh (included in repo). This is a 1-process test, 15 sec.

```
== read 1 ==
8040 : 0
== read 2 ==
8040 : 1838000000
== read 3 ==
8040 : 3528000000
== read 4 ==
logan@Ubuntu22:~/Documents/Cpts360Repo/pa-3-LoganSutton13$
```

```
[ 2405.999432] 8040
                created
[ 2409.370090] procfile read status
[ 2411.369513] PID: 8040        CPU: 0
[ 2416.428398] procfile read status
[ 2417.841000] PID: 8040        CPU: 1838000000
[ 2421.586564] procfile read status
[ 2423.151049] PID: 8040        CPU: 3528000000
[ 2426.598058] procfile read status
[ 2426.616309] KMLAB MODULE UNLOADING
[ 2426.616320] KMLAB MODULE UNLOADED
logan@Ubuntu22:~/Documents/Cpts360Repo/pa-3-Loga
```

Go to Line/Column

From kmlab_test2.sh (included in repo). This is a 2-process test, 10 sec and 15 sec processes.

```
Sutton13/kmlab.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.11.0-19-generic'
gcc -o userapp userapp.c
== read 1 ==
5146 : 0
5147 : 0
== read 2 ==
5146 : 2441000000
5147 : 2432000000
== read 3 ==
5147 : 4006000000
== read 4 ==
logan@Ubuntu22:~/Documents/Cpts360Repo/pa-3-LoganSutton13$
```

```
[ 1831.758765] KMLAB MODULE LOADING
[ 1831.758770] /proc/kmlab created
[ 1831.758771] /proc/kmlab/status created
[ 1831.758772] Timer setup complete
[ 1831.758783] KMLAB MODULE LOADED
[ 1831.766035] 5146
                created
[ 1831.766332] 5147
                created
[ 1834.786745] procfile read status
[ 1836.923994] PID: 5146        CPU: 0
[ 1836.923999] PID: 5147        CPU: 0
[ 1839.984831] procfile read status
[ 1842.045525] PID: 5146        CPU: 2441000000
[ 1842.045531] PID: 5147        CPU: 2432000000
[ 1845.098579] procfile read status
[ 1847.167791] PID: 5147        CPU: 4006000000
[ 1850.108364] procfile read status
[ 1850.121843] KMLAB MODULE UNLOADING
[ 1850.121886] KMLAB MODULE UNLOADED
[ 1874.203955] workqueue: vmstat_shepherd hogged CPU for >10000us 4 time
s, consider switching to WQ_UNBOUND
```